UNIT-II

Cloud Computing Fundamentals

Motivation for Cloud Computing

- The users who are in need of computing are expected to invest more money on computing resources such as hardware, software, networking, and storage; this investment naturally costs a bulk currency to buy these computing resources, all these tasks would add cost huge expenditure to the classical academics and individuals.

- On the other hand, it is easy and handy to get the required computing power and resources from some provider (or supplier) as and when it is needed and pay only for that usage. This would cost only a reasonable investment or spending, compared to the huge investment when buying the entire computing infrastructure. This phenomenon can be viewed as *capital expenditure* versus *operational expenditure*.

- As one can easily assess the huge lump sum or smaller lump sum required for the hiring or getting the computing infrastructure only to the tune of required time, and rest of the time free from that.

- Therefore, **cloud computing** is a mechanism of bringing–hiring or getting the services of the computing power or infrastructure to an organizational or individual level to the extent required and paying only for the consumed services.

Example : Electricity in our homes or offices

Some of the reasons to go for Cloud computing are :

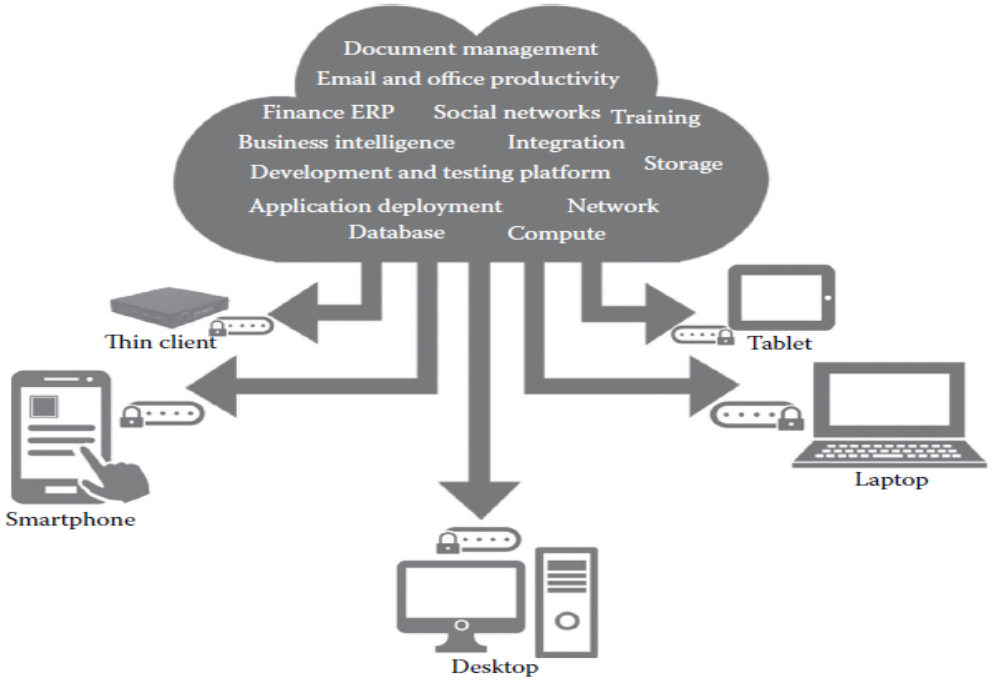
- Cloud computing is very economical and saves a lot of money.

- A blind benefit of this computing is that even if we lose our laptop or due to some crisis our personal computer—and the desktop system—gets damaged, still our data and files will stay safe and secured as these are not in our local machine (but remotely located at the provider's place—machine).

- It is a fast solution growing in popularity because of storage especially among individuals and small- and medium-sized companies (SMEs).

- Thus, cloud computing comes into focus and much needed subscription based or pay-per-use service model of offering computing to end users or customers over the Internet and thereby extending the IT's existing capabilities.

In Figure 2.1 shows several cloud computing applications *cloud* represents the Internet-based computing resources, and the accessibility is through some secure support of connectivity



NIST Definition of Cloud Computing

The formal definition of cloud computing comes from the National Institute of Standards and Technology (NIST): "Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction.

Principles of Cloud computing

The 5-4-3 principles put forth by NIST describe :

(a) Five essential characteristic features that promote cloud computing

(b) Four deployment models that are used to narrate the cloud computing opportunities for Customers while looking at architectural models, and

(c) Three important and basic service offering models of cloud computing

(a)Five Essential Characteristics

Cloud computing has five essential characteristics, which are shown in Figure 2.2. Readers can note the word *essential*, which means that if any of these characteristics is missing, then it is not cloud computing:

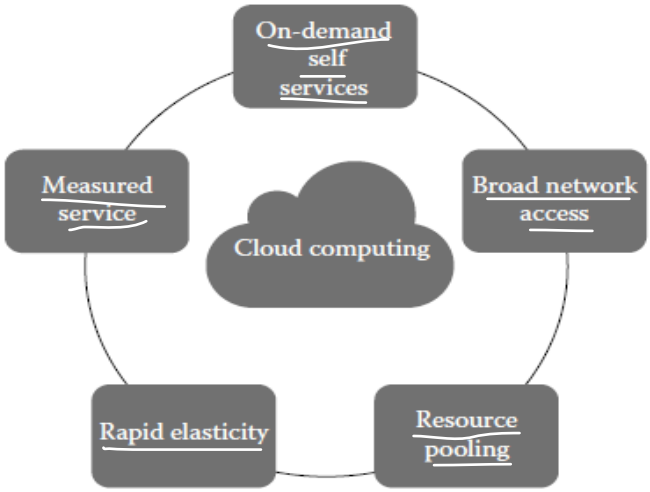


FIGURE 2.2
The essential characteristics of cloud computing.

1. On-demand self-service: A consumer can unilaterally provision computing capabilities, such as server time and network storage, as needed automatically without requiring human interaction with each service's provider.

2. Broad network access: Capabilities are available over the network and accessed through standard mechanisms that promote use by heterogeneous thin or thick client platforms (e.g., mobile phones, laptops, and personal digital assistants [PDAs])

3. Elastic resource pooling: The provider's computing resources are pooled to serve multiple consumers using a multitenant model, with different physical and virtual resources dynamically assigned and reassigned according to consumer demand. There is a sense of location independence in that the customer generally has no control or knowledge over the exact location of the provided resources but may be able to specify the location at a higher level of abstraction (e.g.,country, state, or data center). Examples of resources include storage,processing, memory, and network bandwidth.

4. Rapid elasticity: Capabilities can be rapidly and elastically provisioned, in some cases automatically, to quickly scale out and rapidly released to quickly scale in. To the consumer, the capabilities available for provisioning often appear to be unlimited and can be purchased

in any quantity at any time.

5. Measured service: Cloud systems automatically control and optimize resource use by leveraging a metering capability at some level of abstraction appropriate to the type of service (e.g., storage, processing , bandwidth, and active user accounts). Resource usage can be monitored, controlled, and reported providing transparency for both the provider and consumer of the utilized service.

(b) Four Cloud Deployment Models

Deployment models also called as types of models. These deployment models describe the ways with which the cloud services can be deployed or made available to its customers, depending on the organizational structure and the provisioning location. One can understand it in this manner too: cloud (Internet)-based computing resources—that is, the locations where data and services are acquired and

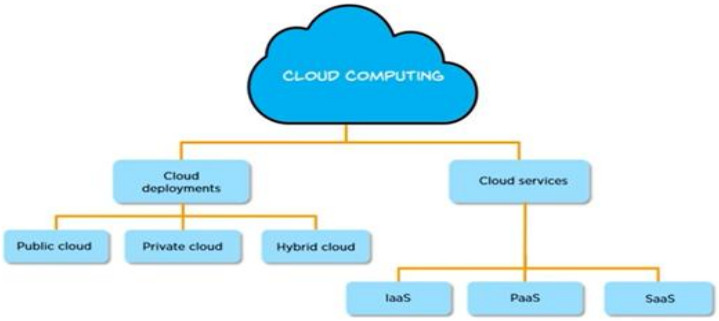
provisioned to its customers—can take various forms. Four deployment models are usually distinguished, namely,

(1) Public

(2) Private

(3) Community, and

(4) Hybrid cloud service usage



Public Cloud

The public cloud makes it possible for anybody to access systems and services. The public cloud may be less secure as it is open to everyone. The public cloud is one in which cloud infrastructure services are provided over the internet to the general people or major industry groups.

Private Cloud

The private cloud deployment model is the exact opposite of the public cloud deployment model. It's a one-on-one environment for a single user (customer). There is no need to share your hardware with anyone else. The distinction between private and public clouds is in how you handle all of the hardware. It is also called the "internal cloud" & it refers to the ability to access systems and services within a given border or organization

Hybrid Cloud

By bridging the public and private worlds with a layer of proprietary software, hybrid cloud computing gives the best of both worlds. With a hybrid solution, you may host the app in a safe environment while taking advantage of the public cloud's cost savings. Organizations can move data and applications between different clouds using a combination of two or more cloud deployment methods, depending on their needs.

Community Cloud

A community cloud is like a shared space in the digital world where multiple organizations or groups with similar interests or needs can come together and use the same cloud computing resources. In a community cloud, these organizations share the same infrastructure, like servers

and storage, but they have their own separate areas within that shared space where they can store and manage their data and applications. This allows them to collaborate and benefit from shared resources while still maintaining some level of privacy and control over their own data.

c) Three Service Offering Models

(i) Infrastructure as a Service

(ii) Platform as a Service

(iii) Software as a Service.

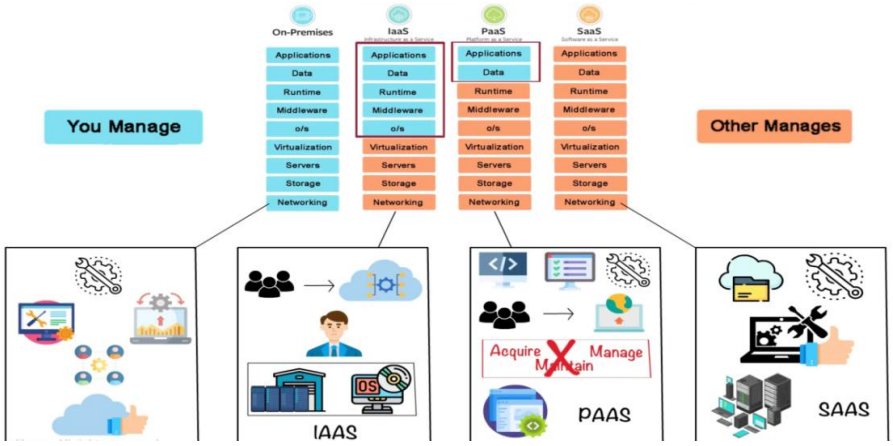
(i) Infrastructure as a Service :- Infrastructure as a service (IaaS) is a service model that delivers computer infrastructure on an outsourced basis to support various operations. Typically IaaS is a service where infrastructure is provided as outsourcing to enterprises such as networking equipment, devices, database, and web servers. It is also known as **Hardware as a Service (HaaS).** IaaS customers pay on a per-user basis, typically by the hour, week, or month. Some providers also charge customers based on the amount of virtual machine space they use.
It simply provides the underlying operating systems, security, networking, and servers for developing such applications, and services, and deploying development tools, databases, etc.

(ii) Platform as a Service:

PaaS is a category of cloud computing that provides a platform and environment to allow developers to build applications and services over the internet. PaaS services are hosted in the cloud and accessed by users simply via their web browser.
A PaaS provider hosts the hardware and software on its own infrastructure. As a result, PaaS frees users from having to install in-house hardware and software to develop or run a new application. Thus, the development and deployment of the application take place **independent of the hardware**.

iii)Software as Service:

Software as a Service (SaaS) is a category of cloud computing that provides software applications over the internet, eliminating the need for users to install or maintain software on their local devices. SaaS applications are hosted in the cloud and accessed by users via a web browser. A SaaS provider manages the infrastructure, security, maintenance, and updates, ensuring that users can focus on utilizing the software without worrying about backend management. This model enables businesses and individuals to use applications on a subscription or pay-per-use basis, reducing upfront costs and simplifying software deployment.



AWS Case study

Organizations had a difficult time finding, storing, and managing all of your data. Not only that, running applications, delivering content to customers, hosting high traffic websites, or backing up emails and

other files required a lot of storage. Maintaining the organization's repository was also expensive and time-consuming for several reasons. Challenges included the following:

- Having to purchase hardware and software components
- Requiring a team of experts for maintenance
- A lack of scalability based on your requirements
- Data security requirements

These issues are solved by AWS S3 ,

- AWS service for storage is S3 Bucket, One of its most powerful and commonly used storage services is Amazon S3. AWS S3 ("Simple Storage Service") enables users to store and retrieve any amount of data at any time or place, giving developers access to highly scalable, reliable, fast, and inexpensive data storage. Designed for 99.999999999 percent durability, AWS S3 also provides easy management features to organize data for websites, mobile applications, backup and restore, and many other applications.
- An Another example of cloud application is a web-based e-mail (e.g., Gmail, Yahoo mail); in this application, the user of the e-mail uses the cloud—all of the emails in their inbox are stored on servers at remote locations at the e-mail service provider.
- However, there are many other services that use the cloud in different ways. Here is yet another example: Dropbox is a cloud storage service that lets us easily store and share files with other people and access files from a mobile device as well.

Elastic resource pooling

Elastic resource pooling is a fundamental concept in cloud computing that refers to the ability to dynamically allocate and share computing resources among multiple users or applications based on demand. It enables efficient utilization of resources by allowing them to be flexibly provisioned and scaled according to workload fluctuations. This concept is central to the scalability, efficiency, and cost-effectiveness of cloud computing environments.

Elastic resource pooling in cloud computing:

1. **Dynamic Allocation**: In traditional on-premises environments, computing resources such as servers, storage, and networking infrastructure are typically provisioned statically, meaning they are allocated in fixed amounts based on predicted peak loads. This approach often leads to underutilization of resources during periods of low demand and potential resource shortages during periods of high demand.

Elastic resource pooling, on the other hand, enables dynamic allocation of resources, allowing them to be provisioned and de-provisioned automatically in response to changing workload requirements.

2. **Shared Resource Pool**: In a cloud computing environment, resources are pooled together into a shared infrastructure that can be accessed by multiple users or applications simultaneously. This shared resource pool includes a variety of resources such as virtual machines (e.g., Amazon EC2 instances), storage (e.g., Amazon S3, Azure Blob Storage), databases (e.g., Amazon RDS, Azure SQL Database), and networking resources (e.g., Amazon VPC, Azure Virtual Network). Users or applications can access these resources on-demand, without having to provision or manage the underlying physical infrastructure.

3. **Scalability**: Elastic resource pooling enables horizontal and vertical scalability, allowing resources to be scaled out (adding more instances) or scaled up (increasing the capacity of existing instances) to meet changing workload demands. This scalability is essential for handling sudden spikes in traffic or processing-intensive tasks without experiencing performance degradation or downtime.

4. **Pay-per-Use Model**: Elastic resource pooling is often associated with a pay-per-use or pay-as-you-go pricing model, where users are charged based on their actual resource consumption rather than a fixed upfront cost. This consumption-based pricing model aligns costs with usage, allowing organizations to scale their infrastructure cost-effectively and pay only for the resources they consume.

5. **Resilience and High Availability**: Elastic resource pooling enhances resilience and high availability by distributing workloads across multiple redundant resources within the shared pool. In the event of hardware failures or infrastructure issues, workloads can be automatically migrated to alternate resources without impacting performance or availability.

Elastic Compute Cloud (EC2)

We know that our computer has a processor, memory (RAM), storage, and runs an operating system like Windows or Linux. You can use it to do various tasks like browsing the internet, running programs, or storing files.

Now, an EC2 instance in AWS (Amazon Web Services) is like having a virtual computer in the cloud. Instead of having a physical machine at your home or office, you rent a virtual computer from AWS.

EC2 instance consists of:

Virtual Hardware: It includes virtual processors, memory (RAM), and storage. You can choose the type and size of this virtual hardware based on your needs. For example, you might need more processing power for running complex calculations or more storage for storing large files.

Operating System: You can choose the operating system you want to run on your EC2 instance, such as Amazon Linux, Ubuntu, Windows Server, etc. This is just like choosing the operating system for your personal computer.

Networking: Each EC2 instance has its own network settings, including an IP address, security groups (firewall rules), and network interfaces. This allows your instance to communicate with other resources in the AWS cloud and the internet.

Purpose: You can use EC2 instances for various purposes, such as hosting a website, running applications, processing data, or even for machine learning tasks. It's like having a versatile computer that you can use for different jobs.

Features of Amazon EC2

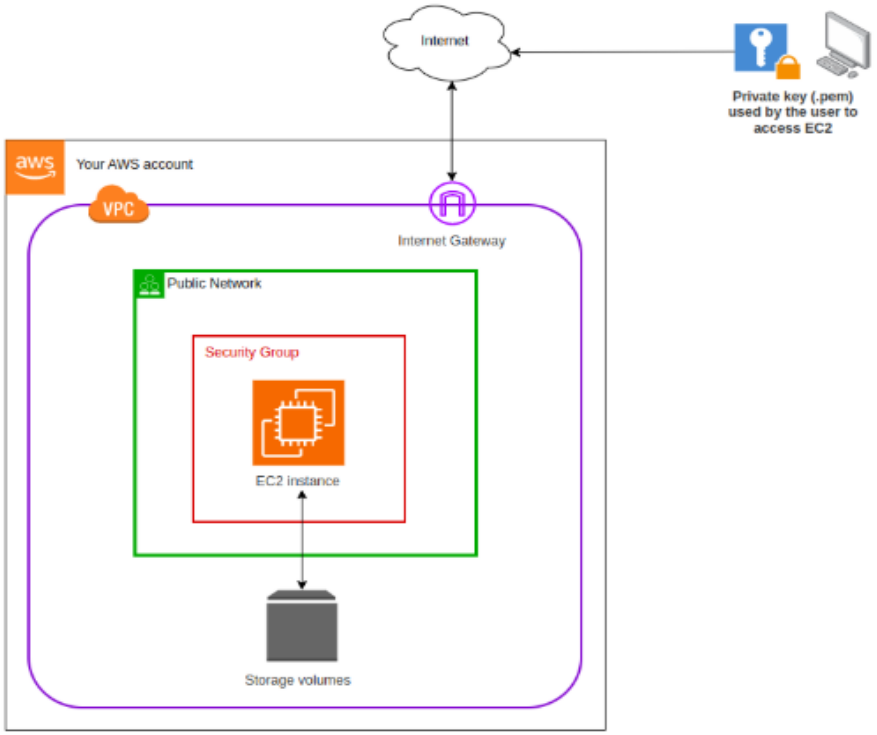
1. Instances: Think of these as virtual computers in the cloud. You can create as many as you need.
2. Tags: These are like sticky notes you can attach to your virtual resources to help you organize and identify them
3. Amazon Machine Images (AMIs): These are like pre-made setups for your virtual computers. They include everything your computer needs to run, like the operating system and any extra software.
4. Instance Types: Imagine these as different models of virtual computers, with varying levels of power and storage space.
5. Key Pairs: It's like having a set of keys to lock and unlock your virtual computer securely. AWS keeps one key (the public key), and you keep the other (the private key).

Note:-Anyone who possesses your private key can connect to your instances, so it's important that you store your private key in a secure place.

6. Virtual Private Clouds (VPCs): Imagine this as your own private corner of the cloud, where you can set up your virtual computers and connect them to your own network. It's like having your own little piece of the cloud just for you.
7. Regions and Availability Zones: Think of these as different neighborhoods and houses in the cloud. You can choose where to place your virtual computers and storage.
8. Amazon EBS Volumes: This is like having a hard drive in the cloud. You can store your data here permanently, even if you turn off your virtual computer

9. Security Groups: These act like a virtual fence around your virtual computers. You can decide who can come in and what they can do.

10. Elastic IP Addresses: It's like having a permanent address for your virtual computer, even if you move it around or turn it off.



Creating an EC2 instance

1. Navigate to EC2:
   -In the AWS Management Console,
   go to the "Services" dropdown.
   - Under "Compute," select "EC2.“

2. Launch an Instance:
   - Click on the "Instances" link in the left panel bar.
   - Click the "Launch Instances" button.

3. Add Name and Tags (Optional):
   - Add any name to your instance for better organization.

4. Choose an Amazon Machine Image (AMI):
   - Select the AMI that best suits your requirements. This is the operating system for your instance

5. Choose an Instance Type:
   - Choose the hardware configuration of your instance (e.g., t2.micro).

6. Select or Create a Key Pair:
   - Choose an existing key pair or create a new one. This is crucial for accessing your instance securely.

   Create the new key pair ,by giving name ->click on create key pair

7. Configure Security Group:
   - Create a new security group or select an existing one. Configure the inbound and outbound rules.

8. Add Storage:
   - Configure the storage settings for your instance. You can increase the default size if needed.

9. Launch Instances:

10. Before launch instance once check with all configurations and number of instance you want

11. Click "Launch Instances."

    Your instance is now being created.

Rapid elasticity using Amazon EBS

Rapid elasticity means that you can quickly scale up or down the amount of storage space you're using. For example, if you suddenly need more storage space because your application is growing, you can easily increase the capacity. Conversely, if you find that you're using more storage space than necessary, you can reduce it just as easily.

Elastic Block Storage (EBS) is like having a virtual hard drive for your computer in the cloud.

Imagine you have a computer where you can store all your files and data.

In the cloud, your computer is virtual, but you still need a place to store your stuff. That's where EBS comes in.

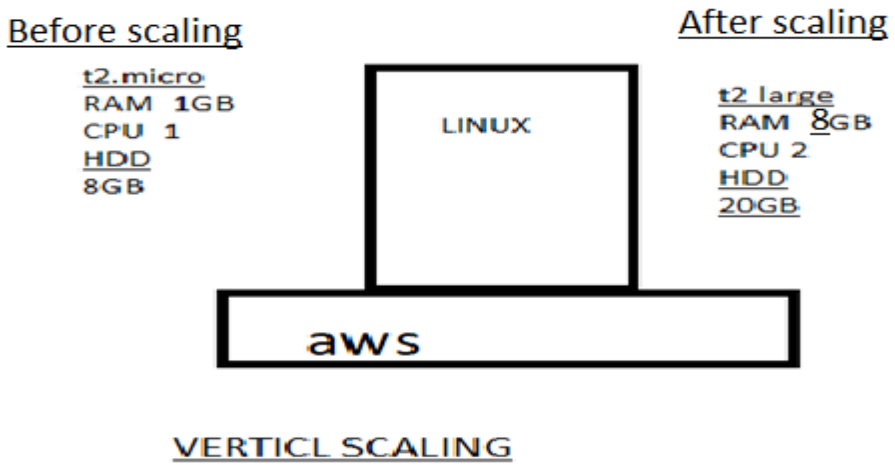
The "elastic" part of EBS means that you can easily adjust the size of your storage volumes as needed. If you need more space, you can simply increase the size of your EBS volume. And if you no longer need as much space, you can decrease it. This flexibility makes it easy to scale your storage up or down depending on your needs.

First we understand the terminology of vertical and horizontal scaling, and then we'll dive into Amazon Elastic Block Store (EBS) in AWS.

1. Vertical Scaling:

Vertical scaling, also known as scaling up or scaling vertically, involves increasing the capacity of a single server or instance by adding more resources such as CPU, memory, or storage. In this approach, you typically upgrade the existing hardware components of the server to handle increased workload or resource requirements. For example, you might add more RAM to a server to improve its performance.

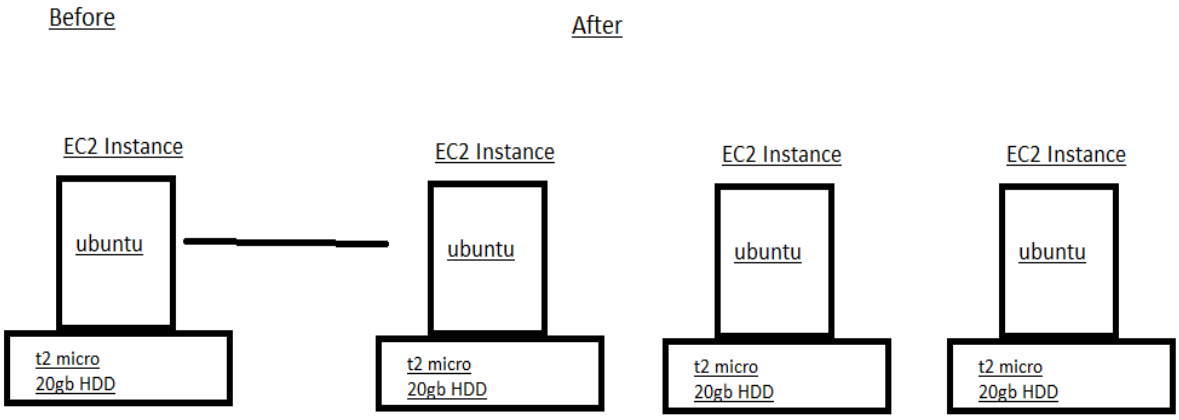
Vertical scaling has its limitations, as there's only so much you can upgrade a single server before reaching its maximum capacity. It also introduces potential points of failure, as all resources are concentrated in a single instance.

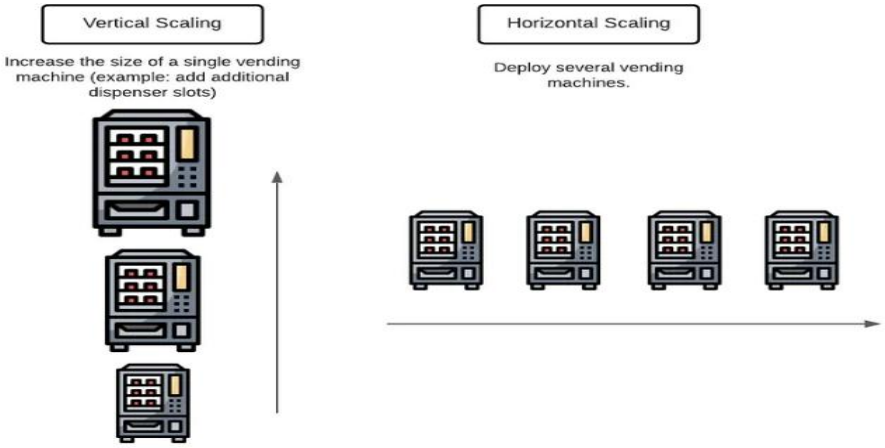


2. Horizontal Scaling:

Horizontal scaling, also known as scaling out or scaling horizontally, involves adding more servers or instances to distribute the workload across multiple machines. Instead of upgrading a single server, you add more servers to your infrastructure to handle increased demand. Each server operates independently and can share the workload with other instances.

Horizontal scaling offers greater scalability compared to vertical scaling, as it allows you to add more resources as needed without being limited by the capacity of a single server. It also provides redundancy and fault tolerance, as multiple instances can continue to operate even if one or more servers fail.





key features and benefits of Amazon EBS:

1. **Elasticity**: With Amazon EBS, you can easily scale your storage capacity up or down based on your application's needs. You can create EBS volumes of different sizes and types (such as General Purpose SSD, Provisioned IOPS SSD, and Throughput Optimized HDD) and attach them to your EC2 instances as needed.

2. **Durability and Availability**: Data stored on EBS volumes is replicated within a single Availability Zone (AZ) to protect against component failure. You can also create snapshots of your EBS volumes to back up your data and restore it in case of data loss or corruption.

3. **Performance**: Amazon EBS offers different types of storage volumes optimized for different performance requirements. For example, General Purpose SSD volumes provide a balance of price and performance for a wide range of workloads, while Provisioned IOPS SSD volumes offer predictable performance for I/O-intensive applications.

4. **Snapshots and Backup**: You can create point-in-time snapshots of your EBS volumes, which are stored in Amazon S3. These snapshots can be used to back up your data, migrate volumes between Availability Zones, or create new volumes.

Note: Auto scaling comes under scale out----------Horizontal Scaling

Vertical Scaling

Create and configure storage services using Amazon EBS

Amazon EBS (Elastic Block Store): Amazon EBS is a **high-performance block storage service** designed for use with EC2 instances. It provides **persistent storage** for virtual machines, meaning data remains available even after the instance is stopped or terminated.
EBS volumes can be attached to EC2 instances and support features like snapshots, encryption, and scalability. Scale Up and Scale Down.

Scale Up /Scale Down (Vertical Scaling):

Definition: Scale up, also known as vertical scaling, involves increasing the resources (such as **CPU, RAM, or disk space**) of an existing single server or virtual machine to handle increased workload or performance requirements.

EBS Equivalent: increasing the **size or performance** specifications of an existing EBS volume

Procedure: increasing/decreasing of CPU and RAM

Step 1. - Navigate to the EC2 dashboard.
Step 2. - Stop the EC2 instance associated with the EBS volume.
Step 3. - Click "Actions" and then—instance settings—change the instance type"Modify Volume".
Step 4. -Increase/ decrease the size or change the volume type to a higher performance specification.
Step 5. - Click "Modify" to apply the changes.
Step 6: Now the start the machine you see with increase /decrease of size and performance

---

Attaching and Detaching Volumes with in same region of EC2

Step 1. **Launching an Instance with Additional Volumes**:
- Start by launching an instance from the EC2 dashboard. Choose your desired AMI (Amazon Machine Image), instance type, and other configurations.

- In the "Add Storage" step, you can add additional volumes. Specify the size and other attributes for these volumes.

Step 2. Attaching a Volume from an Instance:
- Navigate to the EC2 and select "Volumes" under EBS section from the left-hand menu.
- Locate the volume you want to detach from the instance.
- Select the volume, then choose "Actions" -> "Detach Volume".
- Confirm the detachment by clicking "Yes, Detach".

Step 3. Attaching a Volume to Another Instance:

- After detaching the volume, it will become available.
- Select the detached volume from the Volumes list.
- Choose "Actions" -> "Attach Volume".
- Select the instance you want to attach the volume to
- specify the device name (e.g., /dev/sdf), and then click "Attach".
- Once attached, the volume will appear as an additional disk on the chosen instance.

Step 4: Attach and Mount EBS Volume to Linux EC2

Attach and Mount an EBS Volume to a Linux EC2 Instance (Step-by-Step)

Step 1: Create an EBS Volume

1. Log in to **AWS Management Console**
2. Go to **EC2 → Volumes**
3. Click **Create Volume**
4. Choose:
    - **Volume Type**: gp3 (recommended)
    - **Size**: (e.g., 10 GB)
    - **Availability Zone**: **Same AZ as EC2 instance**
5. Click **Create Volume**

Step 2: Attach the EBS Volume to EC2

1. Select the newly created **EBS volume**
2. Click **Actions → Attach Volume**
3. Choose:
    - **Instance**: Select your Linux EC2 instance
    - **Device name**: /dev/xvdf
4. Click **Attach**

Step 3: Connect to the EC2 Instance

Connect using SSH:

ssh -i key.pem ec2-user@<public-ip>

Step 4: Verify the Attached Disk

Check all available disks:

lsblk

You will see:

- Root volume: /dev/xvda
- New volume: /dev/xvdf (no partition yet)

Check filesystem:

lsblk -fs

Step 5: Create a Partition

Create a partition on the new volume:

fdisk /dev/xvdf

Inside fdisk:

- Press m → Help menu
- Press n → New partition
- Press p → Primary partition
- Press Enter → Default partition number
- Press Enter → Default first sector
- Press Enter → Default last sector (use full disk)
- Press w → Write changes and exit

Notify kernel:

partprobe

Step 6: Verify the New Partition

lsblk -fs

You should now see:

/dev/xvdf1

Step 7: Format the Partition

Format with XFS filesystem:

mkfs.xfs /dev/xvdf1

Verify:

lsblk -fs

Step 8: Create a Mount Directory

mkdir /mnt/archana

Step 9: Mount the Volume

mount /dev/xvdf1 /mnt/archana

Verify:

df -h

Step 10: Make the Mount Persistent

Get the UUID of the volume:

blkid /dev/xvdf1

Edit fstab:

nano /etc/fstab

Add this line at the end:

UUID=<UUID-value> /mnt/archana xfs defaults,nofail 0 0

Save and exit.

Test fstab:

mount -a

Step 11: Verify Persistence

1. Reboot the instance:

reboot

2. Reconnect and check:

df -h

Volume should be mounted automatically.

Step 12: Detach and Reattach Volume (Data Recovery Scenario)

Detach:

1. EC2 → Volumes → Select volume
2. Actions → Detach volume

Attach to another EC2 instance:

1. Attach to new instance (same AZ)
2. Login to new instance
3. Create mount directory:

mkdir /mnt/archana

4. Mount the volume:

mount /dev/xvdf1 /mnt/archana

All previous data will be available

Summary Flow

Create Volume → Attach → lsblk → fdisk → mkfs → mkdir → mount → fstab → reboot

---

Attaching Volumes across regions of EC2 using the snapshot

To create a snapshot of an EBS volume and attach it to an instance in another region in AWS, you'll need to follow these steps:

Step 1. Create a Snapshot:

a. Sign in to the AWS Management Console.
b. Navigate to the EC2 Dashboard.
c. Click on "Volumes" in the left-hand navigation pane.

d. Select the EBS volume you want to create a snapshot of.
e. Click on the "Actions" dropdown menu above the volume list and select "Create Snapshot."
f. Provide a name and description for the snapshot.
g. Click on the "Create Snapshot" button to initiate the snapshot creation process.

Step 2. Copy the Snapshot to Another Region:
a. Once the snapshot is created, go to the "Snapshots" section in the EC2 Dashboard.
b. Select the snapshot you just created.
c. Click on the "Actions" dropdown menu above the snapshot list and select "Copy Snapshot."
d. Choose the destination region where you want to copy the snapshot.
e. Click on the "Copy Snapshot" button to initiate the copy process. This may take some time depending on the size of the snapshot and the network speed.

Step 3. Monitor the Snapshot Copy Progress:
a. You can monitor the progress of the snapshot copy by navigating to the "Snapshots" section in the EC2 Dashboard of the source region.
b. Look for the snapshot you copied and check its status. It will change from "pending" to "completed" once the copy process is finished.

Step 4. Switch to the Destination Region:

a. Use the region selector in the top-right corner of the AWS Management Console to switch to the destination region where you copied the snapshot.

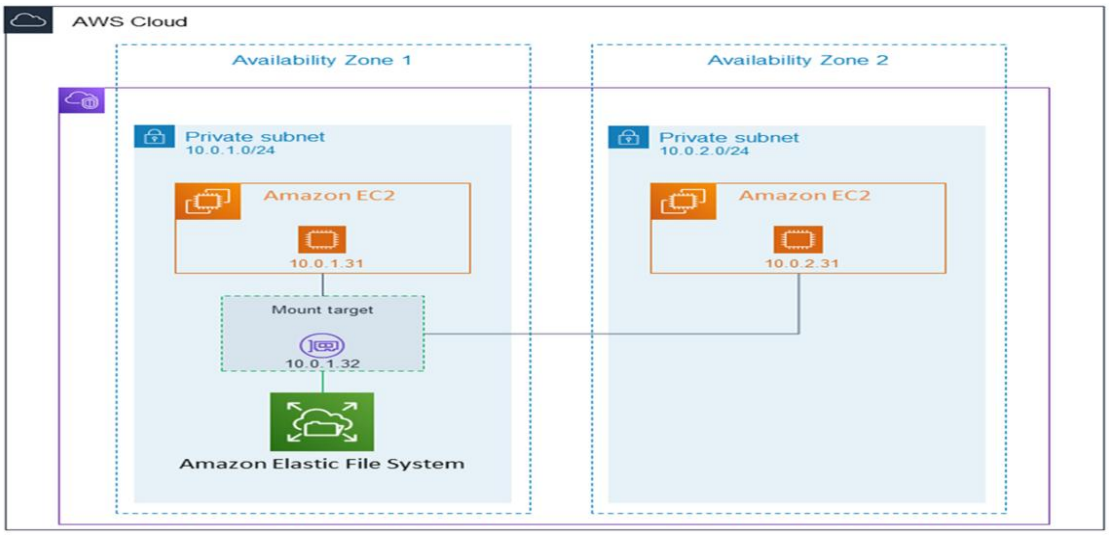
Step 5. Create a Volume from the Snapshot:
a. In the EC2 Dashboard of the destination region, go to the "Snapshots" section.
b. Find the snapshot you copied from the source region.
c. Click on the snapshot, then click on the "Actions" dropdown menu and select "Create Volume."
d. Configure the volume settings, such as volume type, size, and availability zone.
e. Click on the "Create Volume" button to create the volume from the snapshot.

Step 6. **Attach the Volume to an Instance**:
a. Once the volume is created, navigate to the "Volumes" section in the EC2 Dashboard.
b. Find the newly created volume and select it.
c. Click on the "Actions" dropdown menu and select "Attach Volume."
d. Choose the EC2 instance to which you want to attach the volume and specify the device name.
e. Click on the "Attach" button to attach the volume to the instance.

We have created a snapshot of an EBS volume, copied it to another region, created a volume from the snapshot, and attached it to an instance in the destination region.

Elastic File Storage

Amazon Elastic File System (EFS) is a scalable and fully managed file storage service provided by Amazon Web Services (AWS). It allows you to create and configure file systems that can be accessed concurrently from multiple EC2 instances, providing a highly available and scalable storage solution for your applications and workloads.



Amazon EFS supports **for only Amazon Linux instance**:

Step 1: Launching EC2 Instances

1. Sign in to AWS Management Console.

2. Go to the EC2 dashboard.

3. Click on "Launch Instance".

4. Configure instance details:

- Instance Name: EFS-

5. Choose an Amazon Machine Image (AMI) for Linux.

6. Select instance type: t2.micro.

- Key Pair: EFS

- Network: Choose Subnet-1a

- Security Group: **Create a new security group (SG) and add NFS and allow from anywhere.**

7. Launch the instance.

8. Repeat the above steps to launch another instance named **EFS-2 in Subnet-1b** with the same configuration.

Step 2: Creating an EFS File System

1. Go to the EFS service in the AWS Management Console.

2. Click on "Create file system".

3. Specify details:

- Name: Optional

- VPC: Default

- Enable region button.

Click on "Next".

4. In the Network settings:

- Delete all security groups.

- **Select the newly created security group (**NFS).

Click on "Next" and review the configuration.

Click on "Create file system".

Step 3: Accessing the two EC2 instances named EFS-1 & EFS -2 in two different PowerShell sessions and performing the specified tasks:

Accessing EFS-1 Instances in Two Different PowerShell Sessions:

1. Open Two PowerShell Sessions:

- Open two separate PowerShell windows or tabs on your local machine.

For each Instance:

2. SSH into the Instance:

- Use the SSH command to connect to the EFS-1 instance. Replace `[instance-public-ip]` with the public IP address of the instance:

ssh -i [path-to-your-keypair.pem] ec2-user@[instance-public-ip]

3. Switch to Root User:

- Gain root access by executing the following command:

sudo su

4. Create a Directory:

- Make a directory named "efs" using the following command:

mkdir efs

5. Install Amazon EFS Utilities:

- Use yum package manager to install the Amazon EFS utilities:

yum install -y amazon-efs-utils

6. List Files:

- Execute the following command to list files in the current directory:

ls

7. Verify Installation (Optional):

- Optionally, you can verify the installation of the Amazon EFS utilities by checking the version:

efs-utils --version

Repeat Steps 2-7 for Each PowerShell Session Corresponding to EFS-2 Instance.

By following these steps, you'll have accessed each EFS-1 instance in separate PowerShell sessions, switched to the root user, created a directory named "efs," installed the Amazon EFS utilities, and listed files in the directory. This setup allows you to configure and manage each instance individually as needed.

Step 4: Attaching EFS to EC2 Instances

1. Go to the EFS service in the AWS Management Console.

2. Click on the target EFS file system.

3. Click on the "Attach" button.

4. Choose "Mount via DNS" option.

5. Copy the displayed command.

6. SSH into each EC2 instance.

7. Paste and execute the copied command in the terminal to mount the EFS file system onto the instance.

Step 5: Verify and Test EFS

1. Change the directory to EFS on both instances.

2. Create a file on one instance.

3. Verify that the file automatically synchronizes and appears on the other instance.

By above steps, we can successfully create EC2 instances, configured them, created an EFS file system, and attached it to the instances in the same availability zone in the Mumbai region. Now, the instances can seamlessly access and share files stored in the EFS file system.

Simple Storage Service (S3)

Amazon Simple Storage Service (S3) is a cloud-based object storage service provided by Amazon Web Services (AWS). It's designed to store and retrieve any amount of data from anywhere on the web, making it an essential component of many cloud-based applications and services. Here's a detailed explanation of S3:

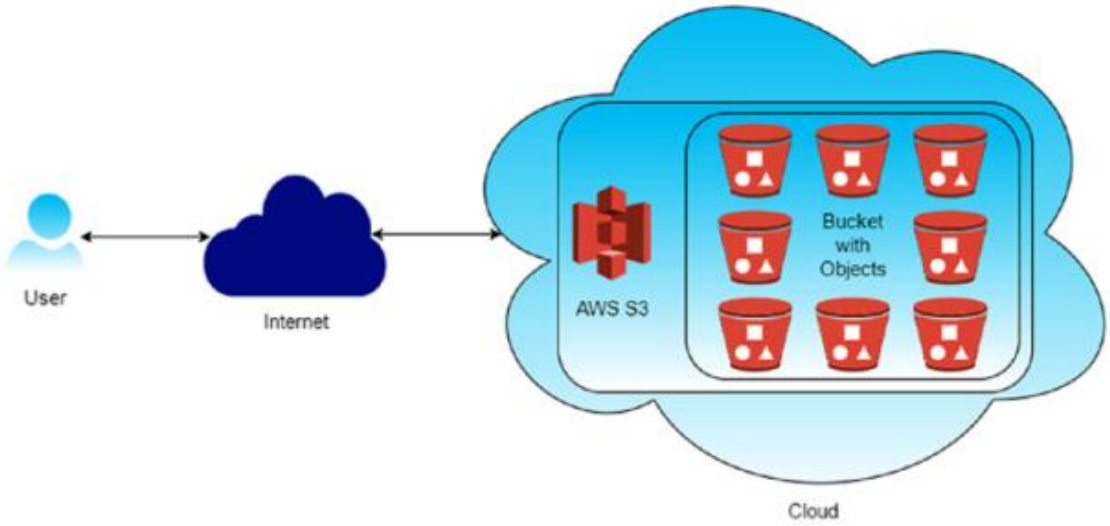
Object Storage:

At its core, S3 is an object storage service. Unlike traditional file systems that store data in a hierarchical structure (folders and directories), S3 organizes data as objects within buckets. Each object consists of the following components:

i. **Key**: A unique identifier for the object within the bucket, similar to a file path.
ii. **Value**: The actual data or content of the object, which can be anything from a document, image, video, to application data.
iii. **Metadata**: Key-value pairs associated with the object, providing additional information such as content type, creation date, or custom attributes.

Buckets:

Buckets are containers for storing objects within S3. They serve as the top-level namespace for organizing and managing objects. When you create an S3 bucket, you must choose a globally unique name, as bucket names must be unique across all of AWS. Each AWS account can have multiple buckets, allowing you to segregate and manage data according to your organizational needs.



S3 Buckets

Features and Capabilities:

S3 offers a wide range of features and capabilities, making it a versatile and reliable storage solution for various use cases:

1. **Durability and Availability**: S3 is designed for high durability, with objects replicated across multiple data centers within a region to ensure data availability and resilience to hardware failures. Amazon S3 is designed for 99.999999999% (11 9's) of durability
2. **Scalability**: S3 is highly scalable and can accommodate virtually unlimited amounts of data, making it suitable for small-scale applications and large enterprises alike.
3. **Security**: S3 provides robust security features, including encryption of data at rest and in transit, access control through IAM policies, bucket policies, and ACLs, and integration with AWS Identity and Access Management (IAM) for fine-grained access control.
4. **Lifecycle Management**: You can define lifecycle policies to automate data management tasks such as transitioning objects to lower-cost storage tiers **(e.g., STORAGE CLASSES- S3 Standard-IA, S3 One Zone-IA, S3 Glacier) or deleting expired data.**

   i. **S3 Standard**

   S3 Standard is the default storage class for Amazon S3. It is designed for frequently accessed data that requires high durability, availability, and low latency.

   ii. S3 Standard-IA (Infrequent Access):

   S3 Standard-IA is a storage class designed for data that is accessed less frequently but requires rapid access when needed.

   It offers the same durability, availability, and low latency as S3 Standard, but at a lower storage cost.

   Objects stored in S3 Standard-IA are ideal for long-term storage and periodic access scenarios.

   iii. S3 One Zone-IA (Infrequent Access):

   S3 One Zone-IA is similar to S3 Standard-IA but stores data in a single AWS Availability Zone rather than across multiple zones.

   It provides cost savings compared to S3 Standard-IA, making it an attractive option for workloads that can tolerate data loss in the event of an Availability Zone failure.

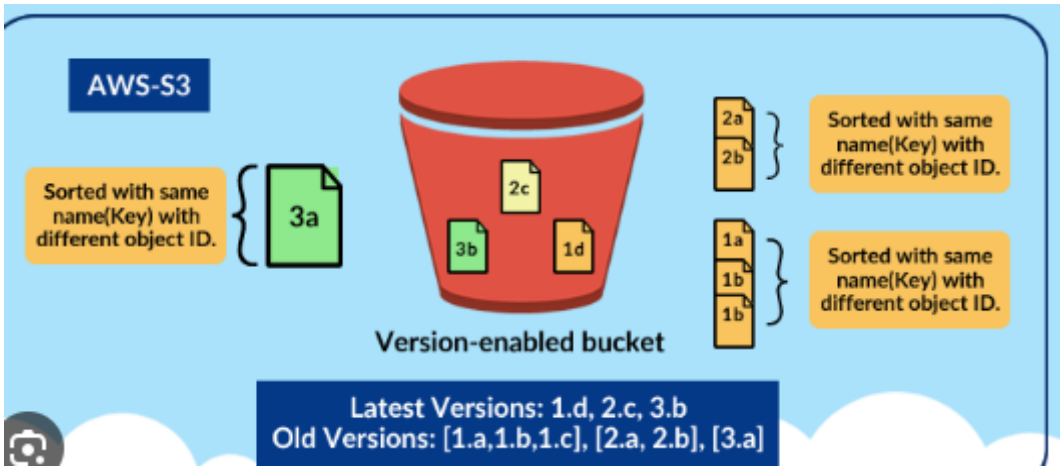
   iv. S3 Glacier:

   S3 Glacier is a low-cost storage class designed for long-term data archival and digital preservation.

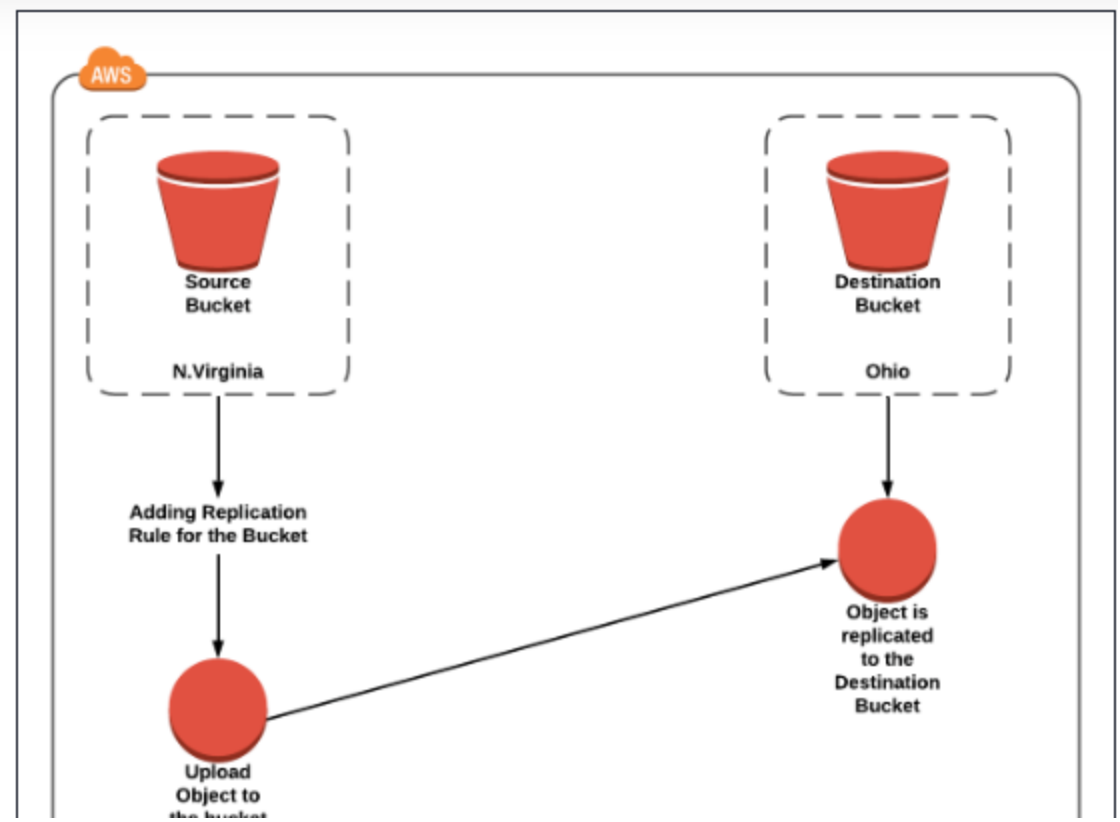
   It offers significantly lower storage costs compared to S3 Standard and S3 Standard-IA, with the trade-off of longer retrieval times.

Objects stored in S3 Glacier are ideal for data that is rarely accessed and can tolerate retrieval times ranging from minutes to several hours.

5. **Versioning**: Versioning in Amazon S3 is a feature that allows you to keep multiple versions of an object in the same bucket. Whenever you upload a new version of an object with the same key (name) as an existing object, S3 doesn't overwrite the existing object but instead creates a new version of it. This means you can retain and access previous versions of objects, providing an additional layer of protection against accidental deletions or overwrites
    i. **Accidental Deletion Protection** :: With versioning enabled, if you accidentally delete an object, you can still retrieve previous versions of it, preventing data loss.
    ii. **Accidental Overwrite Protection**::Versioning prevents accidental overwrites of objects. Even if you upload a new version of an object with the same name, the previous versions are preserved.



6. **Cross-Region Replication**: Cross-region replication (CRR) in Amazon S3 is a feature that automatically replicates objects from one bucket in one AWS region to another bucket in a different AWS region. This provides redundancy and disaster recovery capabilities, ensuring that your data remains available even if an entire AWS region becomes unavailable.
    i. **Disaster Recovery**: Cross-region replication helps ensure business continuity by replicating critical data to a different geographic region, reducing the risk of data loss due to regional disasters or outages.
    ii. **Low-Latency Access**: Users in different geographic regions can access data from the nearest region, reducing latency and improving performance.
    iii. **Compliance**: Some regulatory requirements mandate data replication across multiple geographic regions for data sovereignty and compliance reasons.
7. **Event Notifications**: S3 can trigger events (e.g., object creation, deletion) that can be routed to AWS Lambda, SNS, or SQS for processing.
8. **Integration**: S3 seamlessly integrates with other AWS services such as AWS Lambda, Amazon CloudFront, AWS Glue, Amazon Athena, and Amazon EMR, enabling you to build powerful data processing and analytics workflows.



Amazon S3 Storage Classes

Amazon S3 Storage Classes are different categories of storage options provided by Amazon Simple Storage Service (S3) that are designed to store data based on **access frequency, performance requirements, cost optimization, and availability needs**.
These storage classes enable users to choose the most appropriate storage option for their data by balancing **storage cost, retrieval cost, durability, availability, and access speed**.

By using different storage classes, organizations can **optimize storage expenses**, improve **data management efficiency**, and ensure **high availability and reliability** for frequently accessed data, while providing **low-cost archival solutions** for rarely accessed or long-term data.
Amazon S3 storage classes also support **automated data movement through lifecycle policies**, allowing data to be automatically transferred between storage classes based on usage patterns and business requirements.

i. S3 Standard

- Default storage class for Amazon S3
- Designed for **frequently accessed data**
- High durability (99.999999999%) and availability
- Low latency and high throughput

- Data stored across **multiple Availability Zones**

Use cases:

- Websites
- Mobile applications
- Big data analytics
- Content distribution

ii. S3 Standard–IA (Infrequent Access)

- Designed for **less frequently accessed data**
- Same durability, availability, and performance as S3 Standard
- **Lower storage cost**, but **higher retrieval cost**
- Data stored across multiple Availability Zones

Use cases:

- Backups
- Disaster recovery data
- Long-term storage with occasional access

iii. S3 One Zone–IA

- Stores data in **a single Availability Zone**
- Lower cost than S3 Standard-IA
- Data may be lost if the AZ fails
- Same low latency and high throughput

Use cases:

- Secondary backups
- Easily reproducible data
- Non-critical data

iv. S3 Glacier (Archival Storage)

- Low-cost storage for **long-term archival**
- Not meant for frequent access
- Retrieval time ranges from **minutes to hours**

- Very high durability

Use cases:

- Compliance data
- Archival records
- Digital preservation

Additional Storage Classes

v. S3 Glacier Instant Retrieval

- Archival storage with **millisecond retrieval**
- Higher cost than Glacier but faster access
- Ideal for rarely accessed data that still needs instant access

vi. S3 Glacier Flexible Retrieval

- Formerly called **S3 Glacier**
- Retrieval options:
    - Expedited (minutes)
    - Standard (hours)
    - Bulk (up to 12 hours)
- Very low storage cost

vii. S3 Glacier Deep Archive

- **Lowest-cost** S3 storage class
- Retrieval time: **12–48 hours**
- Designed for long-term retention (years)

Use cases:

- Legal records
- Financial data
- Government archives

viii. S3 Intelligent-Tiering

- Automatically moves objects between access tiers
- No performance impact

- Reduces cost without manual intervention
- Best when access patterns are **unknown or changing**

Comparison of Amazon EBS, EFS, and S3

Feature	**Amazon EBS**	**Amazon EFS**	**Amazon S3**
Full Form	Elastic Block Store	Elastic File System	Simple Storage Service
Storage Type	Block Storage	File Storage	Object Storage
Data Structure	Blocks	Files & directories	Objects (key–value)
Typical Use	OS disks, databases	Shared file systems	Backup, media, logs
Attachment	One EC2 at a time	Multiple EC2s	Accessed via API/URL
File System Required	Yes (ext4, xfs)	No (already managed)	No
Mountable	Yes	Yes	No (API-based)
Persistence	Yes	Yes	Yes
Availability Zone	AZ-specific	Multi-AZ	Regional
Scalability Type	Vertical	Horizontal	Horizontal
Rapid Elasticity	Yes (vertical only)	Yes (horizontal)	Yes (horizontal)
Max Size	Up to 64 TB	Automatically scales	Unlimited
Performance Control	IOPS & throughput	Elastic throughput	Request-based
Access Method	OS-level block	NFS	REST API
Cost Model	Pay per GB + IOPS	Pay per GB used	Pay per object & requests
Durability	High	High	99.999999999% (11 9’s)
Use in Auto Scaling	Limited	Excellent	Excellent

Virtual Private Network (VPC)

A Virtual Private Cloud (VPC) is a virtual network environment in the Amazon Web Services (AWS) cloud that closely resembles a traditional network that you might operate in your own data center.

It allows you to provision a logically isolated section of the AWS cloud where you can launch AWS resources in a virtual network that you define.

VPCs give you control over your network configuration, including IP address ranges, subnets, routing tables, network gateways, and security settings

CIDR Block:

CIDR is a method used to represent IP addresses and specify address ranges in a more flexible and efficient way than traditional IP address classes (Class A, B, and C).

- With CIDR, an IP address is followed by a slash ("/") and a number, known as the prefix length or subnet mask.

- The prefix length indicates the number of bits used for the network portion of the address.

- For example, in the CIDR notation "192.168.1.0/24", the "/24" indicates that the first 24 bits represent the network portion of the address, leaving 8 bits for the host portion.

- For example, you might define a CIDR block of 10.0.0.0/16, which allows for up to 65,536 IP addresses.

Subnets:

- Subnets are segments of the VPC's IP address range where you can place groups of resources, such as EC2 instances, RDS databases, and Lambda functions.

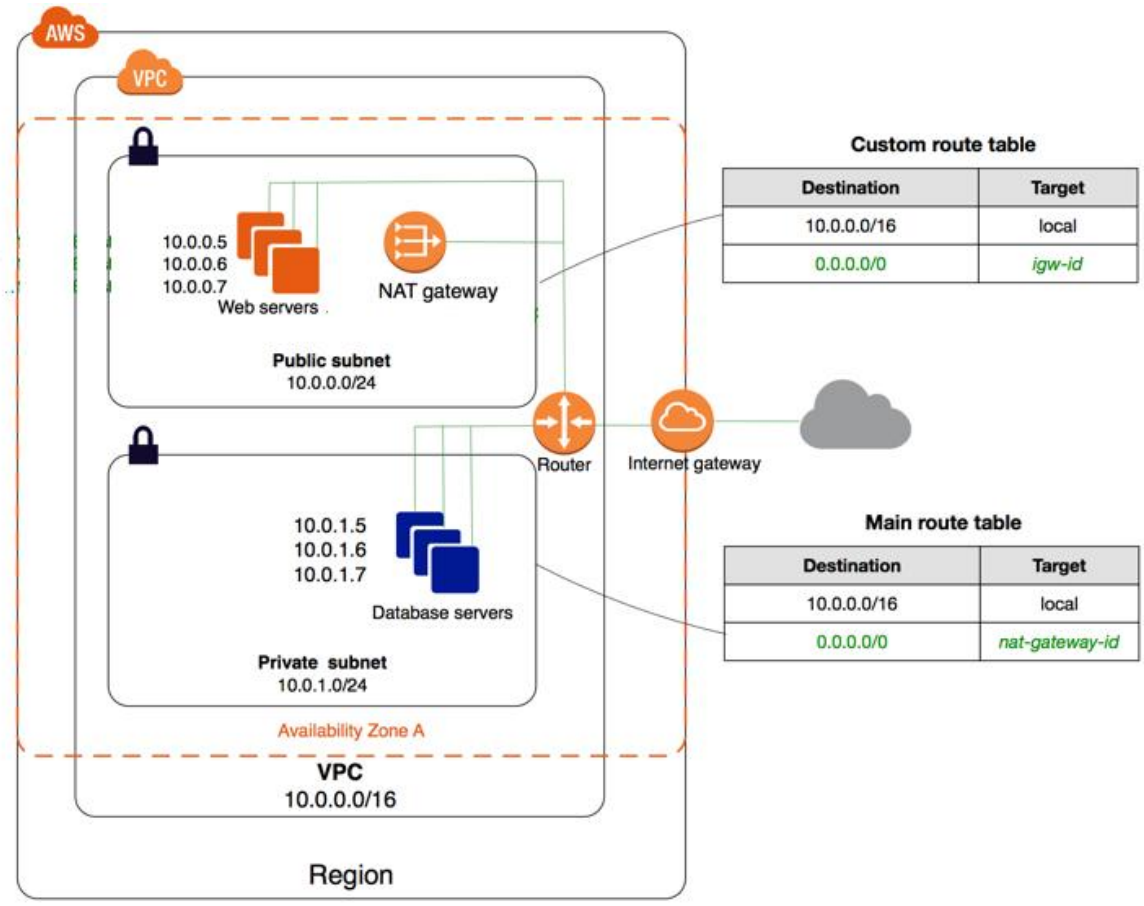
- Each subnet resides in a specific Availability Zone (AZ) and has its own CIDR block, which is a subset of the VPC's CIDR block.

- Subnets serve as a mechanism to isolate resources based on their accessibility and security requirements.

Public Subnet and Web Servers:

The public subnet is designed to host resources that must be accessible from the internet. The web servers deployed in this subnet are assigned private IP addresses.

Although these instances use private IP addresses internally, they can communicate with the internet because the subnet's route table directs internet-bound traffic to the Internet Gateway. This enables the web servers to accept inbound requests from external users and respond accordingly.



Private Subnet and Database Servers:

The private subnet hosts the database servers, which are intentionally isolated from direct internet access to enhance security. Instances in this subnet cannot receive inbound traffic from the internet.

They are accessible only from within the VPC, typically by application servers or web servers in the public subnet, ensuring that sensitive data remains protected.

VPC Router:

AWS automatically provides a virtual router within each VPC. This router is responsible for directing network traffic between subnets and to external gateways according to the rules defined in route tables.

Although the router is not directly configurable by users, its behavior is controlled indirectly through subnet route table associations.

Route Tables

Route tables determine how traffic is routed within the VPC. The public subnet is associated with a custom route table that contains a local route for traffic within the VPC and a default route that sends all internet-bound traffic to the Internet Gateway.

The private subnet is associated with the main route table, which also includes a local route but sends its default internet traffic to the NAT Gateway instead of directly to the Internet Gateway.

This difference in routing behavior is what distinguishes public subnets from private subnets.

Internet Gateway

The Internet Gateway is an AWS-managed component that enables communication between the VPC and the public internet. It is attached at the VPC level and provides a target for route table entries that allow internet traffic. Resources in subnets that route traffic to the Internet Gateway can both send and receive internet traffic, provided security rules allow it.

NAT Gateway

The NAT Gateway is used to provide outbound internet access to instances located in private subnets. It is deployed within the public subnet and has a public IP address.

When private subnet instances initiate connections to the internet, the NAT Gateway translates their private IP addresses into its own public IP address and forwards the traffic through the Internet Gateway.

Responses are returned only for requests initiated from inside the private subnet, preventing unsolicited inbound access.

Internal and External Traffic Flow

Traffic between the web servers and database servers' flows entirely within the VPC using private IP addresses and the local route.

Internet traffic destined for the web servers flows through the Internet Gateway, while outbound internet traffic originating from the database servers flows through the NAT Gateway.

This separation of traffic paths ensures both accessibility and security.

VPC Creation:

Steps to create a VPC (Virtual Private Cloud) network in AWS with one public server and another private server:

Step 1: Create a VPC

1. Sign in to the AWS Management Console.

2. Go to the VPC dashboard.

3. Click on "Create VPC."

4. Enter the following details:

- Name tag: Provide a name for your VPC (e.g., MyVPC).

- IPv4 CIDR block**: Define the IP address range for your VPC (e.g., **10.0.0.0/16**).

5. Click on "Create."

Step 2: Create Subnets

1. In the VPC dashboard, click on "Subnets" in the left-hand navigation pane.

2. Click on "Create subnet."

3. Enter the following details:

- Name tag: Provide a name for your subnet (e.g., PublicSubnet).

- VPC: Select the VPC created in Step 1.

- Availability Zone: Choose an availability zone.

- IPv4 CIDR block: Define the IP address range for the subnet (e.g., 10.0.1.0/24 for a public subnet).

4. Click on "Create."

Note: Every subnet will be by default private.

We want to make one subnet as public.

To make subnet public its two step process.

Step 1: we need to enable public IP

Select the subnet (10.0.1.0/24) --> Actions --> Modify Auto Assing IP Settings

---> Enable Auto Assing public IPV4 Address -- Save

(From now, public IP will be assigned to the machines in this Subnet)

Repeat the above steps to create another subnet for the private server (e.g., PrivateSubnet with an IP address range like **10.0.2.0/24).**

Step 3: Create Internet Gateway (IGW)

1. In the VPC dashboard, click on "Internet Gateways" in the left-hand navigation pane.

2. Click on "Create internet gateway."

3. Attach the internet gateway to your VPC.

Step 4: Update Route Table for Public Subnet

1. In the VPC dashboard, click on "Route Tables" in the left-hand navigation pane.

Now, we need to connect Route table to Subnet.

Select the route table (InternetRT) ---> Subnet Associations tab ---> Edit Subnet Associations ---> Select the subnet (10.0.1.0/24) -- Save

2. Select the route table associated with the public subnet.

3. Click on the "Routes" tab and then "Edit routes."

4. Add a route with destination `0.0.0.0/0` and target the internet gateway created in Step 3.

5. Click on "Save routes."

Step 5: Launch Instances

1. Go to the EC2 dashboard.

2. Launch a new EC2 instance for the public server:

- Select the appropriate AMI (Amazon Machine Image).

- Choose the public subnet created in Step 2.

- Configure security groups to allow inbound traffic on port 80 (HTTP) or other necessary ports.

3. Launch another EC2 instance for the private server:

- Select the appropriate AMI.

- Choose the private subnet created in Step 2.

- Ensure the security group only allows necessary inbound traffic from the public server or other trusted sources.

Step 6: Access Configuration

- The public server will have a public IP address and can be accessed directly over the internet.

- The private server will not have a public IP address and can only be accessed from within the VPC or through a VPN connection.

VPC Peering

VPC Peering is a networking connection between two Virtual Private Clouds (VPCs) that enables you to route traffic between them using **private IP addresses**. It is a foundational AWS networking concept for building secure, multi-tier architectures.

1. Key Characteristics

- **Logical Connection:** It is not a gateway, a VPN, or a physical piece of hardware. It is a logical routing connection that uses the existing AWS infrastructure.
- **Private Traffic:** Traffic stays on the **AWS Global Backbone** and never traverses the public internet, reducing exposure to threats.
- **One-to-One Relationship:** A peering connection is established between exactly two VPCs.

- **High Performance:** There is no single point of failure or bandwidth bottleneck. Speed is limited only by the instance types involved.

2. Critical Rule: Non-Transitivity

VPC Peering is **non-transitive**. This means that if VPC A is peered with VPC B, and VPC B is peered with VPC C, **VPC A cannot communicate with VPC C** through VPC B. To allow communication, you must create a direct peering connection between A and C.

3. Deployment Scenarios

- **Same Account/Region:** Simplest setup for connecting dev/test/prod environments.
- **Cross-Account:** Peering VPCs owned by different AWS accounts (requires a request-acceptance handshake).
- **Cross-Region (Inter-Region):** Connecting VPCs in different geographic locations. All inter-region traffic is automatically encrypted by AWS.

4. Limitations and Constraints

- **Overlapping CIDRs:** You **cannot** peer two VPCs if their IP address ranges (CIDR blocks) overlap. This is the most common cause of peering failure.
- **Quotas:** By default, you can have up to **50 active peering connections** per VPC (expandable to 125).
- **Edge-to-Edge Routing:** You cannot use a peer's Internet Gateway, NAT Gateway, or VPN/Direct Connect connection to reach the internet or on-premises networks.
- **Security Groups:** You can reference a Security Group from a peer VPC only if both VPCs are in the **same region**.

5. Setup Checklist

To make a peering connection work, you must complete these four steps:

1. **Initiate Request:** VPC A sends a request to VPC B.
2. **Accept Request:** The owner of VPC B must accept the request.
3. **Update Route Tables:** In **both** VPCs, you must add a route where the destination is the peer's CIDR and the target is the Peering Connection ID (pcx-xxxxxx).
4. **Update Security Groups:** Ensure the Security Groups on your instances allow inbound/outbound traffic from the peer's IP range.

Role of Bastion server in connecting private subnets of VPC:

Bastion Server: A Bastion Server is a special-purpose EC2 instance placed in a public subnet of an AWS VPC that provides secure access to resources in private subnets. It acts as a gateway/jump box to connect to private instances that cannot be accessed directly from the internet.

Why Bastion?

Private subnet instances in an AWS VPC do not have public IP addresses and therefore cannot be accessed directly from the internet. Direct internet access to private resources is restricted to maintain security. A bastion server is needed because it provides a controlled and secure entry point through which administrators can access private subnet resources without exposing them to the public network.

How It Works?

The user first connects to the bastion server from the internet using secure protocols such as SSH for Linux instances or RDP for Windows instances. Since the bastion server is placed in a public subnet, it is reachable from outside the VPC. After logging into the bastion server, the user can then connect to EC2 instances located in private subnets, as well as databases or other internal services. The access flow is User → Internet → Bastion Host in the public subnet → Private EC2 instances.

Bastion Server and VPC

A bastion server is located in a public subnet within a VPC and is assigned a public IP address or an Elastic IP so that it can be accessed from the internet. It is configured with strict security group rules to allow only limited inbound traffic. The bastion server is used as a secure gateway to access resources that exist within the same VPC, especially those in private subnets.

Advantages of Bastion Server

- Improves security by preventing direct internet access to private subnet resources
- Acts as a centralized access point for managing connections
- Makes auditing and monitoring of access easier
- Cost-effective compared to exposing multiple instances

Disadvantages of Bastion Server

- Acts as a single point of failure unless high availability is configured
- Requires regular maintenance and monitoring
- Still exposed to the internet and can be a target for attacks

Amazon Lambda:

AWS offers various categories of computing services, including

EC2 ------ virtual machines,

Lambda ------serverless compute, and

Elastic Beanstalk------- simplified web application deployment.

In Amazon Web Services (AWS), Lambda is a server less computing service that allows you to run code without provisioning or managing servers.

Lambda is a powerful serverless compute service that enables developers to execute backend tasks in response to events triggered by various sources. Its event-driven architecture, seamless integration with other services,

It lets you run your code in response to events, such as changes to data in an Amazon S3 bucket or an Amazon DynamoDB table, or in response to HTTP requests using Amazon API Gateway **it offers a scalable and efficient way to execute background functions without user interface involvement.**

Traditional web hosting typically involves provisioning physical servers or dedicated servers from a hosting provider. how it generally works without relying on cloud computing.

1. **Server Procurement**: The first step is to acquire physical servers or dedicated servers from a hosting provider. These servers are usually located in data centers managed by the provider.

2. **Server Configuration**: Once the servers are procured, they need to be set up and configured according to the requirements of the website or application. This includes installing the operating system, web server software (such as Apache or Nginx), database server software (such as MySQL or PostgreSQL), and any other necessary software components.

3. **Networking Setup**: Networking configurations, including IP addresses, domain name mapping, firewalls, and security settings, need to be configured to ensure that the servers are accessible over the internet.

4. **Application Deployment**: Once the servers are set up and configured, the website or application files need to be deployed to the server. This involves transferring the files from the development environment to the production server and configuring the web server to serve the application.

5. **Monitoring and Maintenance**: This includes tasks such as monitoring server performance, applying software updates and patches, and troubleshooting any issues that arise.

Traditional web hosting without relying on virtual machines like EC2 requires significant upfront investment in hardware and infrastructure setup, as well as ongoing maintenance and management efforts to keep the servers running smoothly.

While EC2 -virtual machines simplified many aspects of web hosting, what purpose does Lambda serve?

Before getting into the significance of Lambda, let's explore the distinctions between EC2 and Lambda.

EC2 vs. Lambda:

1. **Infrastructure Setup**:

- EC2: Requires setting up the complete infrastructure including servers, operating systems, runtime environments, memory allocation, and network settings.

- Lambda: Requires only creating the Lambda function, and AWS handles the infrastructure automatically. You don't need to worry about server setup, operating systems, or runtime environments.

2. **Software Installation**:

- EC2: Requires installing the required software on the virtual machines manually.

- Lambda: Doesn't require any software installation as AWS manages the runtime environment for you.

3. **Deployment:**

- EC2: You deploy the complete codebase onto the virtual machines.

- Lambda: You deploy snippet code or functions, not the entire application.

4. **Cost Model:**

- EC2: You pay for the resources (compute, storage, etc.) regardless of whether the application is running.

- Lambda: You pay only for the compute time and memory allocated to the function when it's running.

5. **Maintenance and Security:**

- EC2: Requires manual management of firewalls, patches, and security settings. Continuous monitoring is necessary.

- Lambda: AWS takes care of security and scaling automatically. You don't need to manage servers or security patches.

Lambda Triggers:

Lambda functions are triggered by events from various AWS services. Some popular triggers are:

1. **S3 (Simple Storage Service)**: Triggers Lambda functions when objects are created, modified, or deleted in S3 buckets.

2. **CloudWatch**: Triggers Lambda functions based on events or metrics from CloudWatch, such as logs, alarms, or scheduled events.

3. **API Gateway**: Triggers Lambda functions in response to HTTP requests made to API endpoints created with API Gateway.

4. **DynamoDB**: Triggers Lambda functions when there are changes to data in DynamoDB tables.

key features of AWS Lambda:

1. **Serverless Computing**: With Lambda, you can execute your code in response to various events without managing servers. AWS handles the infrastructure provisioning, scaling, and maintenance for you, allowing you to focus on writing code.

2. **Event-Driven Architecture**: Lambda functions are triggered by events from various AWS services, such as changes to data in Amazon S3 buckets, updates to Amazon DynamoDB tables, messages from Amazon SQS queues, or HTTP requests via Amazon API Gateway.

3. **Automatic Scaling**: AWS Lambda automatically scales your application by provisioning the required infrastructure to handle incoming requests. It scales up or down based on the number of requests your function receives.

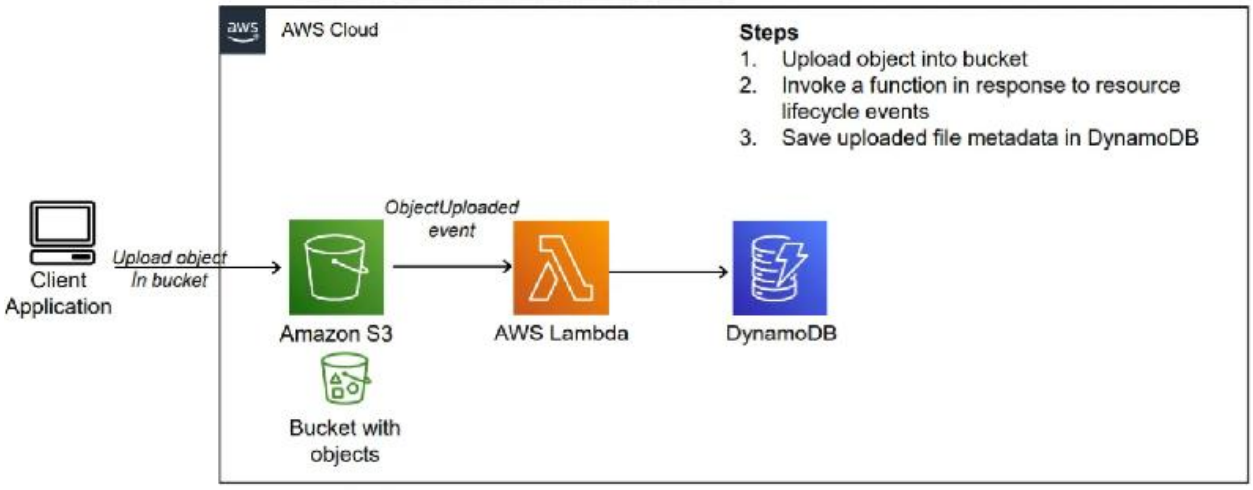
4. **Pay-Per-Use Pricing**: With Lambda, you only pay for the compute time consumed by your code and the number of requests processed, with no charges for idle time. This pay-per-use pricing model offers cost-effective pricing, especially for low-traffic applications.

5. **Support for Multiple Runtimes**: Lambda supports multiple programming languages, including Python, Node.js, Java, Go, Ruby, and .NET Core. This flexibility allows you to choose the runtime that best fits your application requirements and development preferences.

6. **Integration with AWS Ecosystem**: Lambda seamlessly integrates with other AWS services, enabling you to build complex workflows and applications using a combination of serverless services. For example, you can trigger Lambda functions in response to events from Amazon S3, Amazon DynamoDB, Amazon SNS, Amazon SQS, and more.

7. **Built-in Monitoring and Logging**: Lambda provides built-in monitoring and logging through Amazon CloudWatch, allowing you to monitor function invocations, performance metrics, and errors in real-time. You can use CloudWatch Logs to troubleshoot issues and gain insights into your application's behavior.

Case study for lambda riggers



step-by-step procedure for setting up an AWS Lambda function to trigger whenever an object is uploaded to an S3 bucket and update a DynamoDB table:

1. Create an AWS Account: If you haven't already, sign up for an AWS account at https://aws.amazon.com/.

2. Access AWS Management Console: Log in to the AWS Management Console using your AWS account credentials.

3. Open AWS Lambda Console: Navigate to the Lambda service by either searching for "Lambda" in the AWS Management Console or

4. Create a New Lambda Function:

- Click on the "Create function" button.

- Choose "Author from scratch".

- Enter a name for your function (e.g., "S3ToDynamoDB").

- Choose Python as the runtime.

- Under "Permissions", create a new role with basic Lambda permissions or choose an existing role that has permissions to access S3 and DynamoDB.

- Click on the "Create function" button.

5. Upload Code:

- Write the Python code for your Lambda function. This code will be triggered whenever an object is uploaded to the specified S3 bucket.

- Here's an example code snippet:

```
import boto3

from uuid import uuid4

def lambda_handler(event, context):

    s3 = boto3.client("s3")

    dynamodb = boto3.resource('dynamodb')

   # Check if the 'Records' key is present in the event

    if 'Records' in event:

        # Iterate over each record in the event

        for record in event['Records']:

            bucket_name = record['s3']['bucket']['name']

            object_key = record['s3']['object']['key']

            size = record['s3']['object'].get('size', -1)

            event_name = record.get('eventName', 'Unknown')  # Use get() method to handle missing keys
```

```
        event_time = record.get('eventTime', 'Unknown')  # Use get() method to handle missing keys

        dynamoTable = dynamodb.Table('newtable')

        dynamoTable.put_item(

           Item={'unique': str(uuid4()), 'Bucket': bucket_name, 'Object': object_key,'Size': size, 'Event': event_name, 'EventTime': event_time})

    else:

       # Log a message indicating that the 'Records' key is missing from the event

       print("No 'Records' key found in the event.")
```

6. Configure Trigger:

- In the Designer section, click on "Add trigger".

- Select "S3" as the trigger type.

- Configure the trigger to listen to the S3 bucket where you want to trigger the Lambda function.

- Specify the event type (e.g., "All object create events").

- Click on the "Add" button.

7. Configure Environment Variables (**Optional)**:

- If your Lambda function requires any environment variables (e.g., credentials for accessing DynamoDB), you can set them in the "Configuration" tab under "Environment variables".

8. Test Your Function:

- You can test your Lambda function manually by clicking on the "Test" button in the Lambda console.

- You can create a sample test event or use a custom test event to simulate the S3 event that triggers your function.

9. Save and Deploy:

- Click on the "Deploy" button to deploy your function.

10. Monitor Execution:

- You can monitor the execution of your Lambda function in the Monitoring tab of the Lambda console.

- You can also view logs to troubleshoot any issues that may arise during execution.

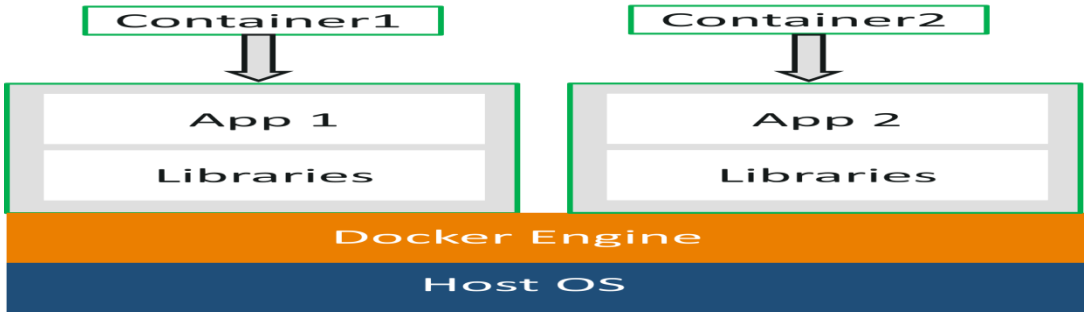
successfully set up a Lambda function to trigger whenever an object is uploaded to an S3 bucket and update a DynamoDB table accordingly.

What is Docker?

Docker is a platform which packages an application and all its dependencies together in the form of containers. This containerization aspect ensures that the application works in any environment.

In the diagram, each and every application runs on separate containers and has its own set of dependencies & libraries. This makes sure that each application is independent of other applications, giving developers surety that they can build applications that will not interfere with one another.

So a developer can build a container having different applications installed on it and give it to the QA team. Then the QA team would only need to run the container to replicate the developer's environment.



Docker Commands

1. **docker –version**

This command is used to get the currently installed version of docker

```
PS C:\Users\user> docker --version
Docker version 20.10.20, build 9fdeb9c
PS C:\Users\user>
```

2. **docker pull**

Usage: docker pull <image name>

This command is used to pull images from the **docker repository**(hub.docker.com

```
PS C:\Users\user> docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
677076032cca: Pull complete
Digest: sha256:9a0bdde4188b896a372804be2384015e90e3f84906b750c1a53539b585fbbe7f
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest
PS C:\Users\user>
```

3. **docker run**

Usage: docker run -it -d <image name>

This command is used to create a container from an image

3. **docker run**

Usage: docker run -it -d <image name>

This command is used to create a container from an image

```
PS C:\Users\user> docker run --name myubuntu -it -d ubuntu
453c8123ec81cf89c7b22e080f707ee9b33971e73d2ecaad1d69cfc801c0ce6b
PS C:\Users\user>
```

4. **docker ps**

This command is used to list the running containers

```
PS C:\Users\user> docker ps
CONTAINER ID   IMAGE     COMMAND       CREATED          STATUS          PORTS     NAMES
453c8123ec81   ubuntu    "/bin/bash"   43 seconds ago   Up 42 seconds             myubuntu
PS C:\Users\user>
```

5. **docker ps -a**

This command is used to show all the running and exited containers

```
PS C:\Users\user> docker ps -a
CONTAINER ID   IMAGE     COMMAND       CREATED              STATUS              PORTS     NAMES
453c8123ec81   ubuntu    "/bin/bash"   About a minute ago   Up About a minute             myubuntu
```

7. **docker stop**

Usage: docker stop <container id>

This command stops a running container

```
PS C:\Users\user> docker stop 453c8123ec81
453c8123ec81
PS C:\Users\user> docker ps -a
CONTAINER ID   IMAGE     COMMAND       CREATED         STATUS                       PORTS     NAMES
453c8123ec81   ubuntu    "/bin/bash"   3 minutes ago   Exited (137) 10 seconds ago            myubuntu
PS C:\Users\user>
```

8. **docker kill**

Usage: docker kill <container id>

This command kills the container by stopping its execution immediately. The difference between ‘docker kill’ and ‘docker stop’ is that ‘docker stop’ gives the container time to shutdown gracefully, in situations when it is taking too much time for getting the container to stop

9. **docker commit**

Usage: docker commit <conatainer id> <username/imagename>

This command creates a new image of an edited container on the local system

10. **docker images**

This command lists all the locally stored docker images

```
PS C:\Users\user> docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
ubuntu        latest    58db3edaf2be   4 weeks ago    77.8MB
PS C:\Users\user>
```

11. **docker build**

Usage: docker build <path to docker file>

This command is used to build an image from a specified docker file

```
PS C:\Users\user\Downloads\fileshare-main\fileshare-main> docker build -t fileshare .
[+] Building 9.5s (4/10)
 => [internal] load build definition from Dockerfile                                0.0s
 => => transferring dockerfile: 389B                                                0.0s
 => [internal] load .dockerignore                                                   0.0s
 => => transferring context: 2B                                                     0.0s
 => [internal] load metadata for docker.io/library/node:latest                      4.3s
 => [auth] library/node:pull token for registry-1.docker.io                         0.0s
 => [1/5] FROM docker.io/library/node@sha256:50b76fc6dc5f03cb3d14c71b8564948aed2bc5124325f35830b2f3be21  5.0s
 => => resolve docker.io/library/node@sha256:50b76fc6dc5f03cb3d14c71b8564948aed2bc5124325f35830b2f3be21  0.1s
 => => sha256:9bf5846b28f63acab0ccb0a39a245fbc414e6b7ecd467282f58016537c06e159 2.21kB / 2.21kB      0.0s
 => => sha256:eaedd085782a535d6194d81e85d728eaeaad930972cda8d14dd8ca95d25fe085 7.51kB / 7.51kB      0.0s
 => => sha256:5c8cfbf51e6e6869f1af2a1e7067b07fd6733036a333c9d29f743b0285e26037 2.10MB / 5.16MB      4.9s
 => => sha256:50b76fc6dc5f03cb3d14c71b8564948aed2bc5124325f35830b2f3be21950ff1 1.21kB / 1.21kB      0.0s
 => => sha256:aa3a609d15798d35c1484072876b7d22a316e98de6a097de33b9dade6a689cd1 8.39MB / 10.88MB     4.9s
 => => sha256:f2f58072e9ed1aa1b0143341c5ee83815c00ce47548309fa240155067ab0e698 2.10MB / 55.04MB     4.9s
 => [internal] load build context                                                   5.0s
 => => transferring context: 2.79MB                                                 4.8s
```

12. **docker push**

Usage: docker push <username/image name>

This command is used to push an image to the docker hub repository

```
PS C:\Users\user\Downloads\fileshare-main\fileshare-main> docker push archanareddycse/fileshare1
Using default tag: latest
The push refers to repository [docker.io/archanareddycse/fileshare1]
f467781874d7: Pushed
b96c3920cda1: Pushed
1a044c6957a0: Pushed
efdadabceebf: Pushed
bd64d88b305d: Mounted from library/node
a5ba4d1f9d2f: Mounted from library/node
d29a2b8b4664: Mounted from library/node
5461a69cb3a7: Mounted from library/node
4ff8844d474a: Mounted from library/node
b7748748ddb: Mounted from library/node
cd247c0fb37b: Mounted from library/node
cfdd5c3bd77e: Mounted from library/node
870a241bfebd: Mounted from library/node
latest: digest: sha256:19aeeeaefdf9a20d4377ddb4b92c4ffa524c4bf6b93db18f279532c591790fa7 size: 3053
PS C:\Users\user\Downloads\fileshare-main\fileshare-main>
```

13. **docker login**

This command is used to login to the docker hub repository

```
PS C:\Users\user> docker login
Login with your Docker ID to push and pull images from Docker Hub. If you don't have a Docker ID, head over to
 https://hub.docker.com to create one.
Username: archanareddycse
Password:
Login Succeeded
```

14. **docker rm**

Usage: docker rm <container id>

This command is used to delete a stopped container

```
PS C:\Users\user> docker ps -a
CONTAINER ID   IMAGE     COMMAND       CREATED        STATUS                      PORTS     NAMES
453c8123ec81   ubuntu    "/bin/bash"   24 hours ago   Exited (137) 24 hours ago             myubuntu
PS C:\Users\user> docker rm 453c8123ec81
453c8123ec81
```

15. **docker rmi**

Usage: docker rmi <image-id>

This command is used to delete an image from local storage

```
PS C:\Users\user> docker rmi 58db3edaf2be
Untagged: ubuntu:latest
Untagged: ubuntu@sha256:9a0bdde4188b896a372804be2384015e90e3f84906b750c1a53539b585fbbe7f
Deleted: sha256:58db3edaf2be6e80f628796355b1bdeaf8bea1692b402f48b7e7b8d1ff100b02
Deleted: sha256:c5ff2d88f67954bdcf1cfdd46fe3d683858d69c2cadd6660812edfc83726c654
PS C:\Users\user>
```

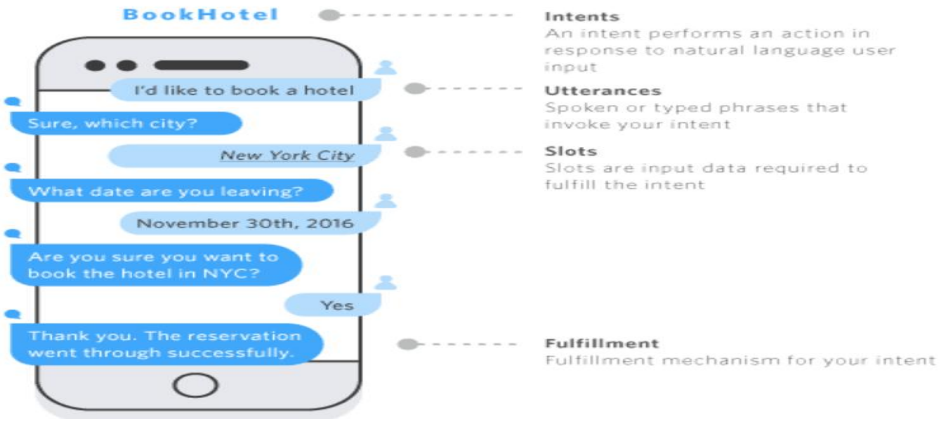
Amazon Lex:

What is LEX

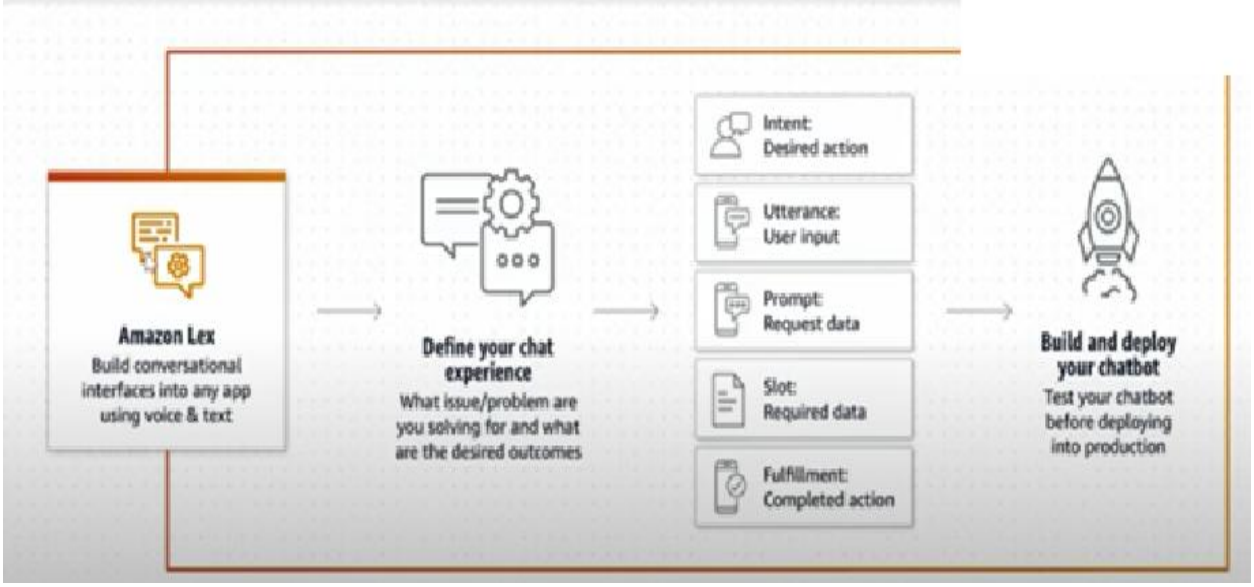
Amazon Lex is a fully managed artificial intelligence (AI) service with advanced natural language models to design, build, test, and deploy conversational interfaces in applications.

Lex is a service that enables developers to build conversational interfaces, commonly known as chatbots or conversational agents, using natural language understanding (NLU) and speech recognition capabilities.

AWS Lex simplifies the process of creating, deploying, and managing chatbots by providing tools and services that handle complex tasks such as language understanding and dialogue management.



Aws Architecture



AWS-LEX

AWS Lex is a service for building conversational interfaces into any application using voice and text, and it involves several key concepts including

- Bot
- Intent
- Utterances
- Initial Response
- Slot
- Slot type
- Confirmation
- Fulfillment
- Closing Response

Bot

A bot is the primary resources type in Amazon Lex . To enable conversations, you add one or more languages to your bot. A language contains intents and slots types.

AWS-INTENT

- An intend is a goal that the user to accomplish through a conversation with a bot.
- You can have one or more related intents, Each intent has sample utterance that convey how the user might express the intent.
- You can create your own intent or choose from a range of pre-defined built-in intents.

AWS-UTTERANCES

- Sample utternces are phrases that represent how a user might interact with an intent to accomplish a goal
- You provide sample utterances and Amazon Lex builds a language model to support interactions through voice and text.
- You can add a slot name to an utterances by enclosing it in braces({}).when you use a slot name ,that slot is filled with the value from the users utterance.

AWS-SLOT

- A slot is information that Amazon Lex needs to fulfill an intent
- Each slot has a slot type.
- Information that a bot needs to fulfill the intent

AWS-SLOT TYPE

- A slot type contains the details that are necessary to fulfill a users intent.
- Each slot has a slot type.
- We can create our own slot type or choose from a range of pre-defined slot types called built-in slot types.
- Some Build types are: Number, Alphanumeric ,date etc

INITIAL –RESPONSE

The initial response is sent to the user after Amazon Lex V2 determines the intent and before it starts to elicit slot values.

CLOSING REPONSE

Configure the bot to respond after the fulfillment with a closing response.

AWS-FULLFILLMENT

Use fulfillment message to tell users the status of fulfilling their intent .You can define messages when fulfillment is successful , and for when the intent cant be fulfilled.

AWS-CONFIRMATION

A confirmation prompt typically repeat back information for the user to confirms . If the user confirms the intent ,the bot fulfills the intent .if the user declines, then the bot responds with a decline response.

Note:

Integration with Other AWS Services:

- Lex seamlessly integrates with other AWS services, allowing developers to leverage additional functionalities for their chatbots. For example, developers can use AWS Lambda functions to implement custom business logic and fulfillment actions triggered by user intents.

Multi-Platform Support:

- Chatbots created with Lex can be deployed across various platforms and channels, including websites, mobile apps, messaging platforms (e.g., Facebook Messenger),